VERSION 1.0 / 9 OCTOBER 2026

RBMI Integrated Library Management System

Operator, administrator and deployment manual

RBCET | RBMI | RBCP | RBCN | RBTTI

Contents

1. Welcome & system scope
2. Local installation & demonstration
3. Live deployment on the free stack
4. Accounts, roles & institution access
5. Catalogue, accessions & barcode labels
6. Patron registration & membership
7. Issue, return, renew & receipts
8. Reservations & hold queue
9. Fine rules, payment & waiver
10. Bulk books & student CSV upload
11. Vendors, orders, invoices & receiving
12. Stock verification & discrepancy review
13. Reports, exports & audit trail
14. Policy settings, holidays & operational controls
15. Backups, restore & go-live acceptance
16. Troubleshooting & CHM compilation

Welcome & system scope

RBMI Integrated Library Management System is a React application with a PostgreSQL backend. It serves RBCET, RBMI, RBCP, RBCN and RBTTI through separate institution workspaces. This manual covers the delivered release, version 1.0, dated 9 October 2026.

The system includes catalogue and physical-copy management, patrons, circulation, holds, fines, bulk CSV upload, acquisitions, stock verification, reporting and audit. It has no public signup screen. Staff and patron accounts are provisioned by administrators.

Read this first

A yellow sample-data banner means you are using a browser-local demonstration. Six mock roles exercise database permissions, but that role selector is not authentication. The sample books and people are fictional. Data is stored in your browser, isolated from other browsers and institutions' live records. A live deployment uses Supabase Auth and PostgreSQL; the sample banner and role selector disappear.

Default policies are starting points, not a claim about RBMI's existing policy: 14-day loans for most categories, 30 days for faculty; 3 copies for most categories, 10 for faculty; 2 renewals; INR 2 per eligible overdue day; 1 grace day; INR 500 cap; block issue/renew above INR 100 outstanding assessed fines; Saturday and Sunday excluded. Every institution/category is configurable.

Terminology

Bibliographic title
: A descriptive record: title, author, ISBN, publisher, subject, DDC/classification and year.

Physical copy
: One accession/barcode, shelf, price and circulation state belonging to a title.

Institution membership
: An Auth user's role in one campus. A user may hold different roles in different campuses.

Assessed fine
: A posted charge created on an overdue return. The dashboard does not estimate unreturned overdue charges.

The date boundary is India time (Asia/Kolkata). This release supports institution-scoped catalogue search; a shared cross-campus OPAC is not included.

Local installation & demonstration

1. Install Node.js 20.16 or newer. A current supported Node LTS is preferred.
2. Extract the ZIP into a local folder. Open PowerShell in the extracted project folder.
3. Run `npm ci` to install the locked dependencies.
4. Run `npm run dev`. Open the Vite URL, normally `http://127.0.0.1:5173`.
5. Keep both Supabase environment variables empty to use sample data. The first load downloads roughly 14 MB of browser PostgreSQL runtime and may take several seconds.

```
npm ci
npm run dev
# Production build
npm run build
npm run preview
```

The Windows shortcut `scripts/start-windows.cmd` checks Node, installs packages and starts Vite. Development uses a server: do not double-click the built index.html through a file URL.

Persist or reset the demo

Records persist in IndexedDB on the same browser origin. Closing the page does not remove them. A different port/origin starts a separate database. In browser Developer Tools, Application/Storage, delete the IndexedDB database `rbmi-library-v1` for this origin, then reload to regenerate the sample. Never enter real patron data into the local demonstration.

Verification commands

```
npm test
npm run build
npm run preview
# Second terminal, with Chrome installed:
npm run test:ui
```

Database tests use the actual PostgreSQL migration and RPCs through PGlite. Browser tests start fresh sample data in headless Chrome. Tests do not prove live Supabase Auth integration, multi-user stress behavior or printer compatibility; verify those in the institution environment.

Live deployment on the free stack

Recommended: Cloudflare Pages for static React assets and Supabase for PostgreSQL, Auth and the Data API. A separate Express/server VM is not required: writes execute in database functions. PostgreSQL supplies relational constraints, reporting queries and transactions for library operations.

1. Create the database

1. Create an RBMI-owned Supabase project in an appropriate region. Keep the database password in an institutional password manager.
2. In Supabase SQL Editor run `supabase/migrations/202610090001_library.sql`, followed by `supabase/seed.sql`, once in an empty project.
3. The seed adds five institutions and category policies only. It does not create sample patrons or default login passwords.
4. Ensure the Data API exposes only the intended public schema; do not expose `app`. The migration removes anon/authenticated direct access to that schema and enables RLS on all its tables.
5. Disable public signup in Auth settings. Create the first operator through Auth > Users.
6. Copy that user's UUID. Replace the placeholder in `scripts/bootstrap-admin.sql` and execute it in SQL Editor. This assigns admin in all five campuses. Choose fewer campuses in that trusted SQL if the operator should manage fewer.

Optional CLI route

Install the Supabase CLI using its official installation instructions. With a Docker-supported local environment, `supabase start` and `supabase db reset` create the local schema and institutional seed. For a remote project:

```
supabase login
supabase link --project-ref YOUR_PROJECT_REF
supabase db push
# Execute supabase/seed.sql in SQL Editor once after initial schema creation.
```

CLI tokens and database passwords must not be committed. This project's config disables signup and uses PostgreSQL 15 for local development. Hosted projects may use a newer supported PostgreSQL version; verify migration compatibility before go-live.

2. Configure React

```
# .env.local (copy .env.example)
VITE_SUPABASE_URL=https://YOUR_PROJECT_REF.supabase.co
VITE_SUPABASE_ANON_KEY=YOUR_PUBLIC_PUBLISHABLE_OR_ANON_KEY
```

Both values must be set together. These are public client configuration, not database credentials. Never place a service-role key, secret key or database password in a VITE variable. A build with both blank deliberately opens the demonstration.

Run the production build and test live sign-in locally. An authenticated user without a membership sees a clear membership error. Use Settings to assign additional users after creating their Auth identities.

3. Publish the frontend

1. Commit the source and lockfile to an RBMI-controlled repository, excluding .env.local and node_modules.
2. In Cloudflare Workers & Pages, create a Pages project from the repository.
3. Build command: `npm ci && npm run build`. Output directory: `dist`. Set Node version to a supported version satisfying package.json.
4. Set both VITE variables in the production build environment. Preview deployments require their own intended environment values. Rebuild after any variable change.
5. Deploy. Set Supabase Auth's Site URL to the resulting HTTPS URL and configure approved redirect URLs if using emailed invitations/password recovery.
6. Configure a custom domain through Cloudflare if desired. Verify HTTPS and that requests to frontend routes return the app.

For direct upload instead of Git integration, build locally with live environment values and run `npx wrangler pages deploy dist --project-name rbmi-library` after Cloudflare authentication. Wrangler may prompt to create the project. The provided wrangler.toml specifies dist.

Free-tier limits, checked 9 October 2026

Supabase Free currently has a 500 MB database, 50,000 monthly active users, 5 GB egress and two active projects; low-activity projects can pause after a week. Automatic database backups are excluded. Cloudflare Pages allows 500 builds/month on Free and free unlimited static requests, subject to platform limits including 25 MiB per asset. This app uses no Pages Functions. Neon announced 1 GB/project free PostgreSQL storage on 2 October 2026, but would require adapting the Auth/API layer. Render Free web services sleep after 15 minutes; no such separate API is needed here.

Research links: Supabase pricing; Cloudflare limits; Neon announcement; Render Free. Keep usage under limits and maintain your own off-site backup. A hosted Sites demonstration is a separate private sample-data deployment; it does not provision the institution's live database.

Accounts, roles & institution access

Live accounts use Supabase email/password authentication. Create or invite a user through the trusted Supabase dashboard. Password resets are administered there; no recovery email workflow is built into this application. Never share one account among operators when audit attribution matters.

Role	Permissions
Admin	All workflows, policies, holidays and institution-role assignments.
Librarian	Catalogue/patrons/imports, desk, fines, acquisitions and stock; read settings.
Assistant	Issue, return, renew and manage holds; read institution records and reports.
Accountant	Record payments/waivers and read records/reports.
Auditor	Read institution records, reports and audit; no writes.
Patron	Read catalogue and own loans/holds/fines; renew own loans and manage own holds.

Assign a membership

1. Select the intended campus using the top-right institution selector.
2. Open Settings > Institution access > Assign role.
3. Enter the existing Supabase Auth user UUID and a role.
4. For a patron role, select that institution's linked patron record. This is required. A staff role may omit the link.
5. Save, then ask the user to sign in again or refresh their session/workspace.

Users only see institutions in their memberships. A user may be admin in one campus and auditor in another. Admin cannot change their own role through this UI. To revoke a membership entirely, a trusted operator uses SQL Editor to delete the appropriate app.memberships row; role reassignment is available in the UI. Preserve at least one authorised administrator.

Patron profiles and Auth accounts are separate: bulk student upload creates library patron records, not passwords or Auth users. Link accounts later when patron self-service is required. Staff read permissions currently include patron contact data; assign roles accordingly.

Catalogue, accessions & barcode labels

1. Open Catalogue and choose Add title.
2. Enter required title and author; add ISBN, publisher, subject, classification/DDC and year when available.
3. Open View copies on the title card.
4. Choose Copy. Enter accession number and barcode; optionally shelf and price.
5. Print the copy's label and affix it to the physical item. Label uses scanner-compatible Code 128.

Each physical copy must have its own barcode. Barcode uniqueness is global; prefix with campus code, for example RBCET-A-2026-001. Accessions are unique within a campus. Adding a bibliographic title alone does not create a copy. Search accepts title, author, ISBN, subject, DDC, accession and barcode.

Edit and change condition

View copies > Edit title updates bibliographic metadata. A copy's Status action allows available, lost, damaged, withdrawn or missing, with a mandatory reason. A currently loaned copy must first be returned before a manual state change. All changes are audited. Copies are retained rather than deleted, preserving loan history.

Withdrawn copies are excluded from a newly started stock session. Lost/damaged/missing copies remain visible for investigation. Returning an item makes it available; if it needs repair, immediately change status to damaged with a reason. Marking a lost copy does not post a replacement charge automatically.

Printer/scanner setup

Use a USB barcode scanner configured as a keyboard wedge and set its suffix to Enter if appropriate. Focus the barcode input before scanning. Use barcode values printable in Code 128 (ordinary ASCII recommended). Label opens a popup; allow popups for this trusted app if the browser blocks it. Print at actual size and test scanning before mass printing.

Patron registration & membership

Open Patrons > Add patron. Required values are patron code, full name and category. Categories are student, faculty, staff, alumni and guest. Optional fields include email, department/class, active flag and expiry date.

Patron codes are unique within each institution. Use stable codes from the student/staff register. Edit keeps the same code to update that membership. Changing the code creates a separate record; the help form warns about this. An inactive patron or a patron whose expiry is before today's India date cannot receive a new issue, renew or place a hold. The expiry date itself remains valid.

The active-loan count shows open loans only. Contact email is informational; no automatic email delivery is configured in this release. Categories select the appropriate circulation policy. If an administrator changes a category, existing loans retain their original policy snapshot and new loans use the new category's policy.

To suspend borrowing, edit Active to unchecked and save. Do not remove records with loan history. For a graduating student, review active loans and assessed fine balances before suspending membership and issuing an institutional clearance outside the software.

Issue, return, renew & receipts

Issue

1. Use Issue a book on Overview or Circulation.
2. Scan/type the copy barcode or campus accession.
3. Enter the patron code and Save changes.
4. Confirm the loan appears with the expected due date.

The server checks copy availability, patron active/expiry status, configured policy, maximum open loans, overdue loans, assessed outstanding fine threshold and first waiting hold. A copy cannot have more than one open loan. A loaned, lost, damaged, withdrawn or missing copy is blocked.

Due dates

Base due date is India today plus the policy's loan_days. When weekends are excluded, Saturday/Sunday are skipped for the due date. Listed holidays are always skipped. The date moves forward until an open day. A calendar with more than 366 consecutive closed dates is rejected.

Renew

Choose Renew on an active loan. A patron may renew only their own. Renewal fails when overdue, inactive/expired, at the renewal cap, above the assessed-fine threshold, or when any waiting hold exists on that title. A successful renewal adds the original policy's loan_days to the later of current due date and today, then moves to an open day.

Return

Choose Return on an active loan. The operation closes the loan, restores copy availability and assesses a fine if applicable in one transaction. Repeating a return fails without another charge. The message shows the assessed amount. Returned loans remain in the history. Choose Receipt to print a return slip with patron, book, barcode, return date and fine.

Return has no patron-self-service action; it requires an authorised desk role. If a network result is unclear, refresh and inspect the loan before retrying. Returns and policy violations are checked in PostgreSQL even when an interface is bypassed.

Reservations & hold queue

Open Holds > Place hold, or View copies > Place hold. Choose a bibliographic title and enter the patron code. For patron self-service use the linked profile's code. Active and unexpired membership is required. A patron may have only one waiting hold per title.

Waiting holds are served in creation order, with UUID as the stable tie-breaker for equal timestamps. The first waiting patron has priority over every available copy of that title. If another patron tries to issue it, the server reports that it is reserved. Issue to the first patron changes that hold to fulfilled. Subsequent copies can serve the next waiting patron.

Any waiting hold blocks renewals on the title, including a hold by the same borrowing patron. Cancel a waiting hold using the Cancel action. Patrons can cancel only their own. Fulfilled/cancelled records remain visible in history.

This release does not send notification messages or enforce automatic hold pickup expiry. Librarians monitor the queue and cancel stale requests according to institution policy. Holds can be placed even when copies are available, allowing desk-managed reservation workflows.

Fine rules, payment & waiver

Calculation

On return, eligible dates are each day strictly after due date up to and including the return date. Calendar holidays are excluded. Saturday/Sunday are excluded when the loan's policy says so. Subtract grace_days from the eligible-day count, floored at zero; multiply by fine_rate; apply fine_cap. Due-date day is never overdue. A return on or before the due date has zero fine.

```
fine = min(cap, max(0, eligible overdue days - grace days) * rate)
```

Example: due Friday 2 October 2026, returned Friday 9 October. Weekend exclusion plus Monday 5 October as a holiday leaves Tuesday-Friday: 4 eligible days. With 1 grace day and INR 2/day, fine is INR 6. A cap of INR 3 reduces it to INR 3.

Issue-time policy is saved with the loan: later rate/cap/grace edits do not change it. Holidays use the institution's current calendar at return; adding a missed holiday can alter the calculation of still-open loans. Assessed posted fines do not recalculate after a calendar change. The outstanding balance considers posted fines only; open overdue loans independently block new issue.

Collect a fine

1. Open Fines and locate the patron's charge.
2. Choose Payment. Enter a positive amount up to the outstanding balance.
3. Enter a receipt number or collection reference and Save.
4. Confirm the transaction in Payments & waivers.

This records desk collection; it does not debit a bank/card. Part payments are allowed. Overpayment is rejected by the function and database constraints.

Waive

Choose Waive; enter an amount and approval/reason. Admin, librarian and accountant can perform finance actions. Every waiver appends a ledger entry and audit reason. Balance equals assessed amount minus paid minus waived. No silent deletion or negative fine adjustment is exposed. Export transactions for financial reconciliation.

Bulk books & student CSV upload

1. Open Catalogue or Patrons and choose Bulk upload.
2. Download the corresponding template.
3. Save your data as UTF-8 CSV with a header row, maximum 1,000 rows and 5 MB per file.
4. Select the file. Review validation errors and the first eight rows in the preview.
5. Check the review confirmation and Import.
6. Confirm the success count. On an error, no row from that batch is committed.

Book columns

Required: title, author, accession, barcode. Optional: isbn, publisher, subject, classification, year, shelf, price. One row represents one physical copy. If a non-empty ISBN already exists in the same institution, its bibliographic record is reused and the incoming metadata does not overwrite it. Blank ISBN makes a new title record; use the manual title/copy workflow to attach copies to an existing no-ISBN title.

```
title,author,isbn,accession,barcode,shelf,price
Learning React,Alex Banks,9781492051725,A2026-01,RBCET-A2026-01,C-2,650
```

Student/patron columns

Required: code, name. Optional: email, category, department, active, expires. Blank category defaults to student. active accepts true/false; blank defaults to true. expires uses YYYY-MM-DD or blank. An existing campus patron code updates the record. Blank expiry clears a previous expiry; blank active reactivates. Include those fields explicitly when updating an existing register.

```
code,name,email,category,department,active,expires
RBCET-S101,Example Student,,student,BTech Year 1,true,2027-06-30
```

CSV quoting follows normal CSV rules: quote fields containing commas/newlines/double quotes, double embedded quote characters. Preserve codes/ISBNs as text in Excel to prevent lost leading zeros. Client validation is a convenience; the PostgreSQL transaction remains authoritative. Existing barcode/accession conflicts or invalid data reject the entire batch. Uploaded student records do not create Auth accounts.

CSV exports prefix spreadsheet-formula-like text values with an apostrophe to reduce formula execution when opened in spreadsheet software.

Vendors, orders, invoices & receiving

1. Create the bibliographic title in Catalogue first.
2. In Acquisitions, add a vendor with name and optional contact details.
3. Create a purchase order, selecting vendor/title, invoice or order reference, quantity and unit price.
4. When the shipment is accepted, choose Receive.
5. Enter a unique accession prefix and shelf. The app creates PREFIX-1 through PREFIX-N and campus-prefixed barcodes in one transaction.
6. Print labels from Catalogue and check the received status.

The invoice/order reference is unique per vendor/title in the campus. Multiple titles can share one invoice reference. Received value is quantity multiplied by unit price; tax, discounts and multi-line invoice accounting are not modelled. Receive accepts the full ordered quantity only. Partial receipts are not supported.

A received order cannot be received again. Duplicate accession/barcode failures roll back the copies and preserve ordered status. Cancel applies only to an ordered acquisition. Library records and vendor contacts remain for audit/reporting.

The acquisitions module supports acquisition recordkeeping, not vendor emailing, electronic purchase approval or bank payments.

Stock verification & discrepancy review

1. Open Stock verification > Start session and give it a name, such as Annual stock 2026.
2. The system snapshots all non-withdrawn copies and their current states. Only one open session per campus is allowed.
3. Choose Scan copy and scan a barcode. Repeat across shelves. Unknown barcodes are rejected.
4. Review seen, unscanned and on-loan-at-snapshot results.
5. Export the session CSV. Close when scanning is complete.

A copy loaned at snapshot time is reported as on loan if not seen, rather than a shelf absence. Unscanned non-loaned copies are investigation candidates; previously lost/damaged/missing states remain explicit in the expected-status column. A scan repeated in the same session refreshes seen time, not an extra inventory item.

New acquisitions after the snapshot are outside that session. Movements after start do not alter the expected snapshot. Closing is read-only reconciliation: it never automatically marks copies lost/missing or charges patrons. Investigate discrepancies, then use Catalogue > copy Status with a reason for confirmed condition changes. Closed sessions cannot be scanned again.

Reports, exports & audit trail

Overview shows total physical copies and titles, current loans, overdue loans and outstanding assessed fines. Collection status summarises copy states. Loans needing attention lists overdue copies.

Reports exports the collection register, circulation history, overdue register, fine balances, patron register and acquisitions. Most borrowed titles counts all historical issues per title. The institution selector limits exports to the selected campus. Patron role has no staff report module; own records remain accessible through Circulation/Holds/Fines.

Tables show the first 30 records with Show more for further records. CSV exports include all loaded records. Nested policy objects in loan exports are serialised as JSON text; authoritative policy snapshots remain in the database/loan history. Exports are data extracts, not a full database backup.

Audit

Every successful data-changing action records campus, Auth actor UUID, action, timestamp and submitted details. View Details shows the JSON payload. Fine waiver reasons, copy status reasons and imports are visible. Failed operations abort without a success audit entry. Authentication events are available separately in Supabase logs.

Application users cannot edit/delete audit records through the client. Database owners retain administrative database access. Bulk imports record both per-row actions and batch details, increasing database usage. There is no automated retention purge in this release; plan retention under institutional policy and back up before any trusted administrative archival.

Policy settings, holidays & operational controls

Only an admin can save policies, add/update holidays or assign institution memberships. Librarians and other staff may view settings. Each campus has a distinct policy row for each category.

Setting	Meaning
loan_days	Calendar days added before moving the due date to an open day.
max_loans	Maximum open loans for a patron.
max_renewals	Allowed renewals per loan; zero forbids renewal.
fine_rate	INR per eligible overdue day.
grace_days	Eligible fine days waived before the rate is applied.
fine_cap	Maximum assessed fine per returned loan; zero means no fine, not unlimited.
block_balance	Borrowing blocked only when assessed outstanding fines are above this value; equality is allowed.
exclude_weekends	Exclude Saturday/Sunday from fine days and move due dates away from those days.

Policies are snapshotted at issue, including renewal limits. Test a new issue after changing a policy. A changed category policy does not rewrite existing loans.

Holidays

Add the campus date/name. Saving the same date updates its name; there is no delete button in this release. To remove an erroneous date, the trusted database administrator deletes that institution/date from app.holidays via SQL Editor, documenting the maintenance externally. Add holidays before issuing/returning whenever possible. All configured holidays are fine-exempt even if weekends are counted.

Day-to-day controls

Use named operator accounts, confirm the campus before bulk upload, review patron-update semantics, and reconcile collection receipts. Do not automatically resubmit a payment after a network timeout; refresh the ledger first to avoid double recording. Use browser refresh/data refresh after another operator changes records.

Backups, restore & go-live acceptance

Database backups

Supabase Free currently has no automatic database backup entitlement. Schedule an institution-owned backup outside this app, encrypt it and keep it off-site. Browser CSV exports are not a complete backup. Store database credentials outside command arguments/source; use an approved PostgreSQL password file with appropriate permissions or a password manager integration.

```
# Configure connection and secure credential handling first.
# pg_dump custom format preserves schema, constraints and data.
pg_dump --format=custom --no-owner --file=rbmi-backup.dump YOUR_APPROVED_CONNECTION
# Restore only into a prepared isolated test project, never over production blindly.
pg_restore --no-owner --no-acl --dbname=YOUR_TEST_CONNECTION rbmi-backup.dump
```

Coordinate Supabase-specific Auth/storage backup requirements with the current Supabase documentation. A database dump alone may not cover all platform configuration or external storage. This release stores no uploaded file assets or book covers. After restore, check Auth identities, memberships, policy rows, copy states, open loans, fines and audit. Keep migration history and deployed source/version separately.

Acceptance checklist

1. Create named accounts for admin, librarian, assistant, accountant, auditor and patron. Verify allowed actions and rejected actions.
2. Verify each campus sees only its own records; patron accounts see only their own profile/loans/holds/fines.
3. Issue a barcode, reject a duplicate issue, return it, renew within the cap and reject after the cap.
4. Check an inactive/expired patron, maximum loans, overdue block and assessed-fine balance block.
5. Test FIFO hold priority and renewal blocking.
6. Test weekend/holiday/grace/cap fine examples agreed by RBMI.
7. Collect a part payment and waiver, verify receipts, reason and audit; reject overpayment.
8. Import valid books/students; verify a conflicting row rolls the full batch back.
9. Receive an order once and scan/close a stock session; inspect missing-vs-loaned distinctions.
10. Test two simultaneous operators and an unknown-result network interruption; verify constraints and operator recovery.
11. Print and scan labels using your actual devices; print a return receipt.
12. Restore an encrypted backup to a test environment and compare key record counts.

Only promote real institutional data after these checks. The shipped local engine tests do not replace a hosted multi-connection load test. For a large catalogue or growing audit history, add server pagination and measure query/memory/egress before capacity becomes a problem.

Troubleshooting & CHM compilation

Symptom	Resolution
Demo appears in a live deployment	Both build-time environment values were blank. Set them in the correct hosting environment and rebuild.
No institution membership	Create Auth user and assign their UUID via bootstrap SQL/Settings. Refresh sign-in.
Copy not available / reserved	Check copy state and the first waiting hold. Do not bypass the database rules.
Import fails with unique violation	Review existing accessions/global barcodes. Entire batch rolls back; correct and retry.
Backend unreachable	Check project URL/public key, network, Supabase project status/Free pause and service status. Do not enter production data in demo as a workaround.
Stale dashboard	Use Refresh. There is no realtime subscription in this release.
Transaction deadlock	PostgreSQL rejected one conflicting request; refresh and retry only the rejected operation.
Blank/broken demo	Use an up-to-date browser with IndexedDB/WebAssembly enabled. Serve through HTTP(S), not file://. Reset sample IndexedDB if an old incompatible demo schema persists.
Barcode popup blocked	Allow popups for the app; labels should contain ordinary printable barcode values.

Windows CHM source

The delivered help project contains RBMI.hhp (project/options/file list), RBMI.hhc (contents tree), RBMI.hhk (keyword index), standalone HTML topics and local CSS assets. It does not need the React server or a database to read.

1. Install Microsoft HTML Help Workshop from an approved Microsoft source. Its compiler executable is hhc.exe, distinct from the Windows hh.exe viewer.
2. Open PowerShell in the project directory.
3. Run `.\scripts\build-chm.ps1` or pass the full compiler path with `-Compiler`.
4. Open the newly generated docs/help/RBMI.chm in Windows. Test Contents, Index, Search and internal links.

```
.\scripts\build-chm.ps1 -Compiler 'C:\Program Files (x86)\HTML Help Workshop\hhc.exe'
```

The build script requires a newly produced nonempty output; it does not assume an exit code alone proves success. If Workshop is unavailable, use index.html/printable.html and your browser's Print > Save as PDF. A compiled CHM was not created in the build environment because no compiler was installed.

If a downloaded CHM opens with blocked content, extract it to a local trusted folder and use Windows file Properties > Unblock if available. Do this only for the help file you trust. HTML topics remain an alternative on platforms without CHM support.